Lista de exercícios

Questão 1

Uma plataforma de streaming precisa decidir se um usuário pode assistir a determinado conteúdo. A regra considera idade, assinatura, disponibilidade regional e classificação indicativa.

```
None
def can_watch_content(user, content):
    if (
        user["subscription"] == "Premium"
        and user["age"] >= content["minimum_age"]
        and content["available"]
        and user["country"] in content["available_countries"]
    ):
        return True

    return False
```

Sua tarefa: analise a condição utilizada e refatore o código para facilitar a compreensão das regras de acesso.

Pense em como alguém que nunca viu esse código conseguiria entender cada condição sem precisar decifrar toda a expressão.

Questão 2

Um sistema de monitoramento recebe registros de acesso e precisa identificar possíveis tentativas suspeitas.

```
None
def analyze_access(log):
    if (
        log["failed_attempts"] >= 5
        and log["account_status"] == "active"
        and log["ip_address"] not in log["trusted_ips"]
        and log["hour"] < 6
    ):
        print("Suspicious access detected")
```

```
        return "ALERT"

    print("Access considered normal")
    return "NORMAL"
```

Sua tarefa: analise a expressão condicional e a organização das informações utilizadas.

O código precisa continuar tomando exatamente a mesma decisão, mas a regra de identificação de acesso suspeito deve ficar mais fácil de interpretar.

Questão 3

Um sistema de comércio eletrônico possui a seguinte estrutura:

Arquivo `product.py`:

```
None
class Product:
    def __init__(self, name, price, weight, stock):
        self.name = name
        self.price = price
        self.weight = weight
        self.stock = stock
```

Arquivo `shipping.py`:

```
None
def calculate_shipping(product, distance):
    if product.weight <= 1:
        weight_fee = 5
    elif product.weight <= 5:
        weight_fee = 12
    else:
        weight_fee = 20

    distance_fee = distance * 0.30

    return weight_fee + distance_fee
```

Arquivo `checkout.py`:

```
None
from shipping import calculate_shipping


def finalize_purchase(product, distance):
    shipping_cost = calculate_shipping(product, distance)
    total = product.price + shipping_cost

    print("Product:", product.name)
    print("Shipping:", shipping_cost)
    print("Total:", total)

    return total
```

Sua tarefa: analise a distribuição das responsabilidades entre os arquivos.

Considere onde a regra de cálculo do frete deveria estar localizada e se a estrutura atual representa bem a responsabilidade de cada módulo.

Atenção: não basta simplesmente copiar a função para outro arquivo. Pense também nas chamadas que utilizam essa função.

Questão 4

Um sistema de processamento de documentos possui três módulos.

Arquivo `document.py`:

```
None
class Document:
    def __init__(self, title, content, author):
        self.title = title
        self.content = content
        self.author = author
```

Arquivo `document_export.py`:

```
None
def generate_filename(document):
    filename = document.title.lower().replace(" ", "_")
    author = document.author.lower().replace(" ", "_")
```

```
    return f"{filename}_{author}.txt"
```

Arquivo `export_service.py`:

```
None
from document_export import generate_filename


def export_document(document):
    filename = generate_filename(document)

    with open(filename, "w", encoding="utf-8") as file:
        file.write(document.content)

    print("Document exported:", filename)

    return filename
```

Sua tarefa: analise se a função `generate_filename()` está no módulo mais adequado.

Considere quais informações ela utiliza, qual é sua responsabilidade e qual módulo deveria concentrar esse comportamento.

Questão 5

Uma aplicação de análise de dados recebe informações sobre vendas.

Arquivo `sales.py`:

```
None
def calculate_average_sales(sales):
    total = sum(sales)
    return total / len(sales)


def calculate_high_sales(sales):
    return [sale for sale in sales if sale > 1000]
```

Arquivo `sales_report.py`:

```
None
from sales import calculate_average_sales, calculate_high_sales


def generate_sales_report(sales):
    average = calculate_average_sales(sales)
    high_sales = calculate_high_sales(sales)

    print("Average sales:", average)
    print("High sales:", len(high_sales))

    return {
        "average": average,
        "high_sales": high_sales
    }
```

Sua tarefa: avalie a organização dos módulos e das funções.

Observe a relação entre as operações realizadas e os dados utilizados. Analise se existe alguma função que estaria melhor posicionada em outro módulo.

Não presuma que todas as funções precisam ser movidas.